La IA no conoce tu proyecto.

Reglas y skills: lo más barato que pueden hacer para que les entregue mejor código.

Con

Carlos René Angarita Sanguino

Elian Camilo Angarita Sanguino

Este prompt lo acabamos de lanzar dos veces, a la misma IA:

> Crea una app web para registrar los gastos de caja menor de una oficina. Debe permitir agregar un gasto (fecha, descripción, categoría y valor), ver la lista y el total por categoría.

```
~/demo-dkd/a-sin-nada     carpeta vacía
~/demo-dkd/b-configurado  un archivo de reglas y una skill
```

En una hora vemos qué salió.

Primero, ustedes. ¿Cómo usan la IA hoy?

Herramienta	ChatGPT, Copilot, Cursor, Claude. ¿Desde el navegador, el editor o la terminal?
Última vez	¿Qué le pidieron, qué salió y qué tuvieron que corregir?
Lo que más fastidia	Eso que hace que a veces prefieran escribirlo ustedes.
Tiempo de hoy	¿Cuánto les toma un endpoint, una pantalla o un CRUD típico?

La IA es un dev senior que llega nuevo cada mañana y no se acuerda de nada.

Sabe programar. No sabe nada de su proyecto, su stack ni sus acuerdos. Lo que no le dicen, lo adivina, y adivina lo más genérico.

Chat, editor o agente: cambia cuánto ve de su código.

DÓNDE	VE EL PROYECTO	CORRE COMANDOS	EJEMPLOS
Chat en el navegador	No, copian y pegan	No	ChatGPT, Claude.ai
Editor	Los archivos abiertos	Poco	Copilot, Cursor
Agente	**Todo el repo**	**Tests, build, git**	**Claude Code, Codex**

Terminal o app de escritorio: es la misma herramienta con otra cara.

Antes de seguir: el agente corre comandos en su máquina.

01	Permisos en modo aprobar. Nada de saltarse todos los permisos.
02	Ningún secreto, token ni clave en los archivos de reglas.
03	Revisar el diff antes de cada commit.
04	El código lo firma quien hace el commit, no la IA.

Un archivo en la raíz del repo que la IA lee al empezar cada conversación.

Es el onboarding. Lo escriben una vez y deja de tocarles repetir lo mismo.

Cada herramienta le pone su nombre. La idea es la misma.

ARCHIVO	LO LEEN
`AGENTS.md`	**Codex, Cursor, Copilot, Claude Code y la mayoría**
`CLAUDE.md`	Claude Code
`.github/copilot-instructions.md`	GitHub Copilot
`.cursor/rules/`	Cursor
`GEMINI.md`	Gemini CLI, que se puede apuntar a AGENTS.md

Para un equipo con varias herramientas: **un solo AGENTS.md**. Si alguien necesita algo solo para Claude, un CLAUDE.md con la línea `@AGENTS.md`.

Lo que hoy le repiten a mano, va en el archivo.

Sí va	No va
Stack y versiones	Lo que ya hace el linter
Comandos: correr, testear, lint	“Escribe buen código”
Acuerdos que no son obvios	Más de 200 líneas
Lo que nunca debe hacer	Secretos

Este es el archivo de la carpeta B.

Lo mismo que le explicarían a alguien nuevo el primer día. Se escribe en 20 minutos.

b-configurado/AGENTS.md

```
# Caja menor
App interna para una oficina en Colombia.

## Stack
- React 18 + Vite + TypeScript estricto.
- Tests con Vitest + Testing Library.
- CSS plano. Sin librerías de UI. Datos en localStorage.

## Comandos
- npm test: deben pasar antes de terminar.

## Convenciones
- Lógica en funciones puras, cada una con test.
- Interfaz en español de Colombia.
- Pesos con Intl.NumberFormat('es-CO', COP).
- Fechas AAAA-MM-DD. Todo campo con su label.

## Nunca
- Agregar dependencias sin preguntar.
```

Dos niveles: lo del equipo y lo de cada uno.

Proyecto	En la raíz del repo, en git. Lo comparte todo el equipo.
Personal	En su carpeta de usuario: idioma, tono, cómo le gusta que le expliquen.

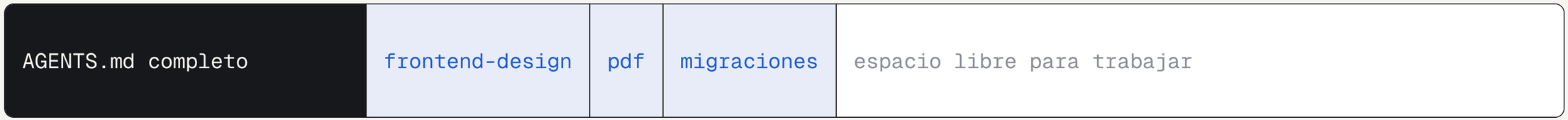
`/init` lee el repo y escribe un primer borrador. Se edita, no se acepta tal cual.

Parte 2 · Skills

Manuales que la IA abre solo cuando la tarea los necesita.

Una carpeta con un SKILL.md: instrucciones, ejemplos y scripts para una tarea específica.

Lo que la IA carga al empezar una conversación.



Reglas: siempre, enteras. Por eso cortas.

Skills: solo nombre y descripción, hasta que hacen falta.

Cuando la tarea es de diseño, abre frontend-design completo. Las demás ni las lee. Por eso una skill puede ser larga sin estorbar.

Hoy se instala una. Crear las propias viene después.

SKILL.md es un formato abierto: Claude Code, Codex, Cursor, Copilot y Gemini CLI lo leen. Cambia la carpeta, no el archivo.

Ojo: una skill de terceros es código ajeno. Se lee antes de instalar, como cualquier dependencia.

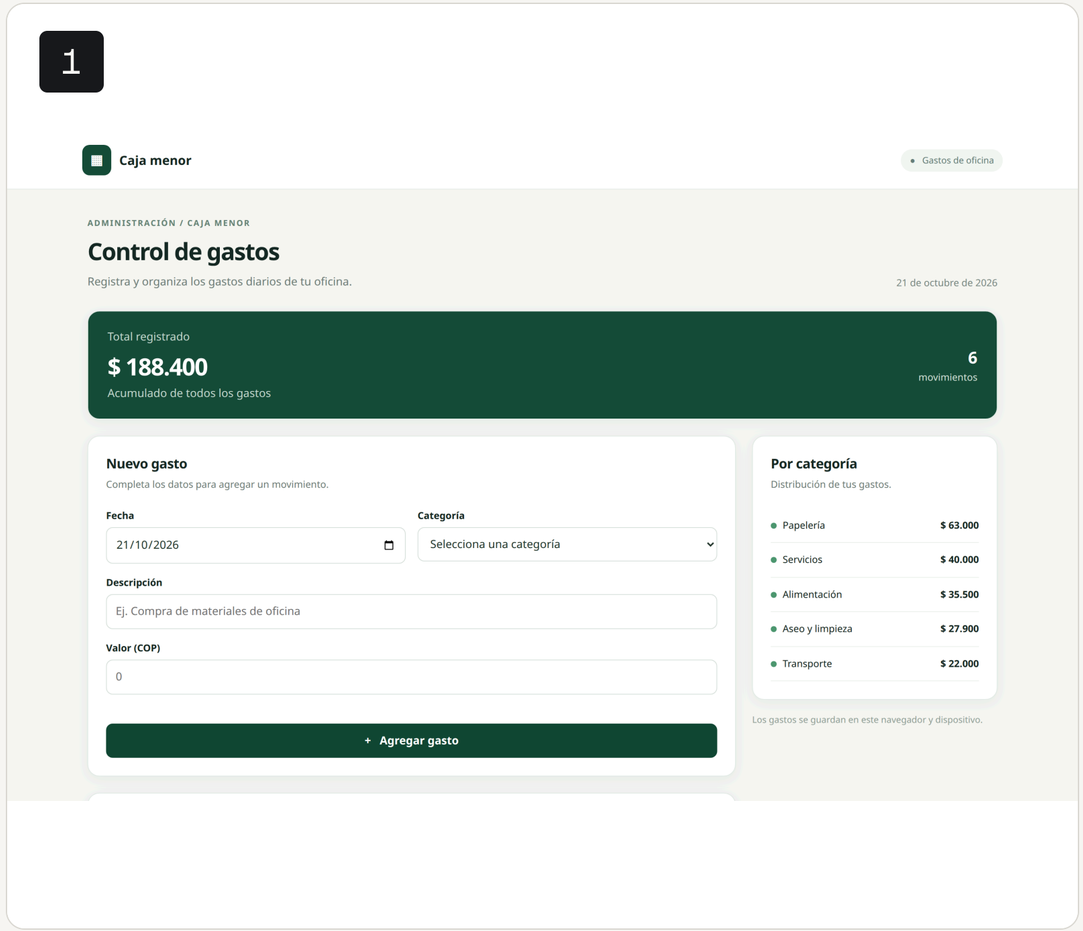
.claude/skills/frontend-design/SKILL.md

```
---
name: frontend-design
description: Guidance for distinctive,
  intentional visual design when building
  new UI or reshaping an existing one.
---

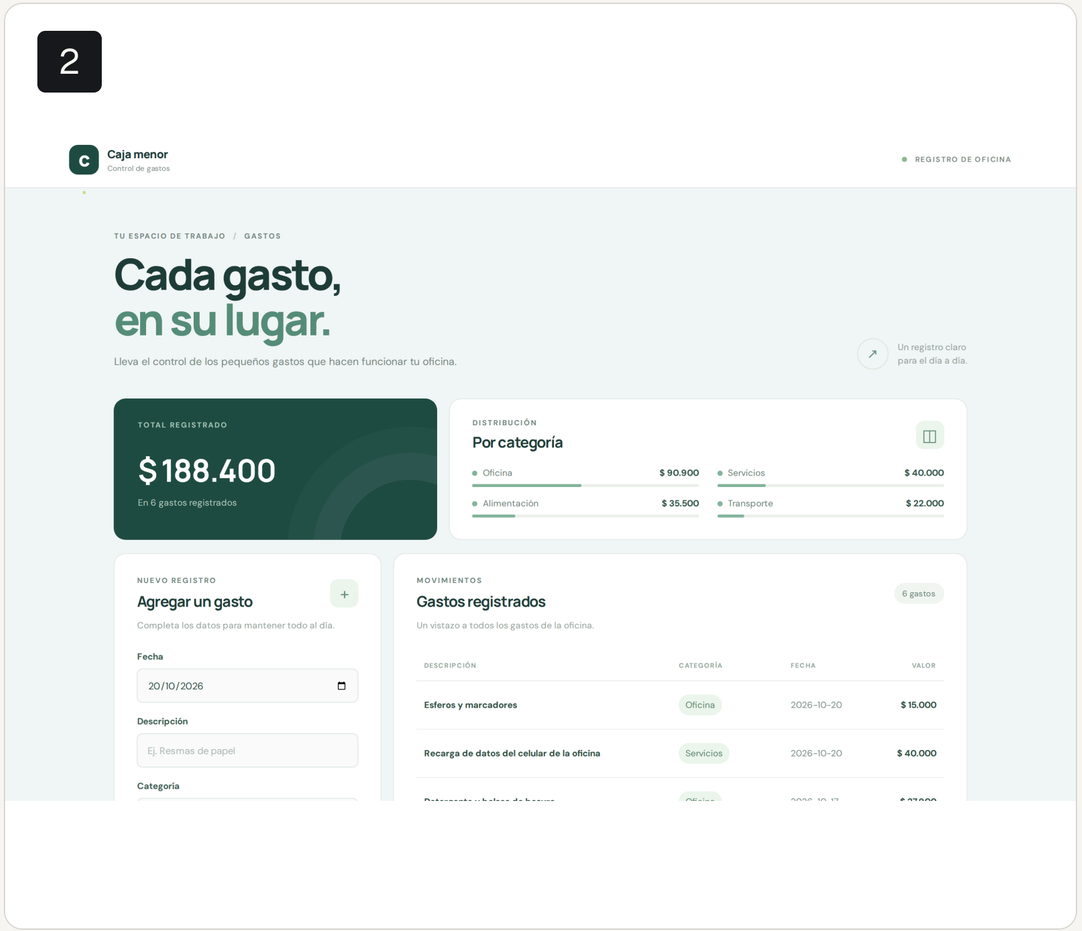
# Frontend Design
Approach this as the design lead at a
small studio known for giving every
client a visual identity...
```

¿Cuál le entregarían a un cliente?

1



2



La revisamos entre todos.

CRITERIO	CÓMO SE MIRA	A	B
Funciona sin tocar nada	`npm run dev`		
Tiene tests y pasan	`npm test`		
Lógica separada y probada	abrir src/		
Pesos y fechas bien	un gasto de 15000		
Sin dependencias de más	`package.json`		
Accesible con teclado	tabular el formulario		

Mismo modelo. Mismo prompt. Dos archivos de diferencia.

Una corrida es una anécdota. Esta semana lo comprueban en su propio código.

20 minutos, en su repo.

1. Abran su proyecto real y saquen el borrador con /init o con la plantilla.
2. Dejen solo lo útil y agreguen 3 reglas que hoy le repiten a mano.
3. Pídanle una tarea pequeña de verdad y miren si las respeta.

¿Solo Claude Code? CLAUDE.md. ¿Otra herramienta o equipo mixto? AGENTS.md.

Para la próxima: usen su archivo y anoten dos cosas.

01	Una vez en que la IA no respetó una regla.
02	Algo que tuvieron que repetirle y que debería estar en el archivo.

Próxima sesión: por qué la IA “se olvida” de cosas en conversaciones largas, y cuánto cuesta eso en tokens.